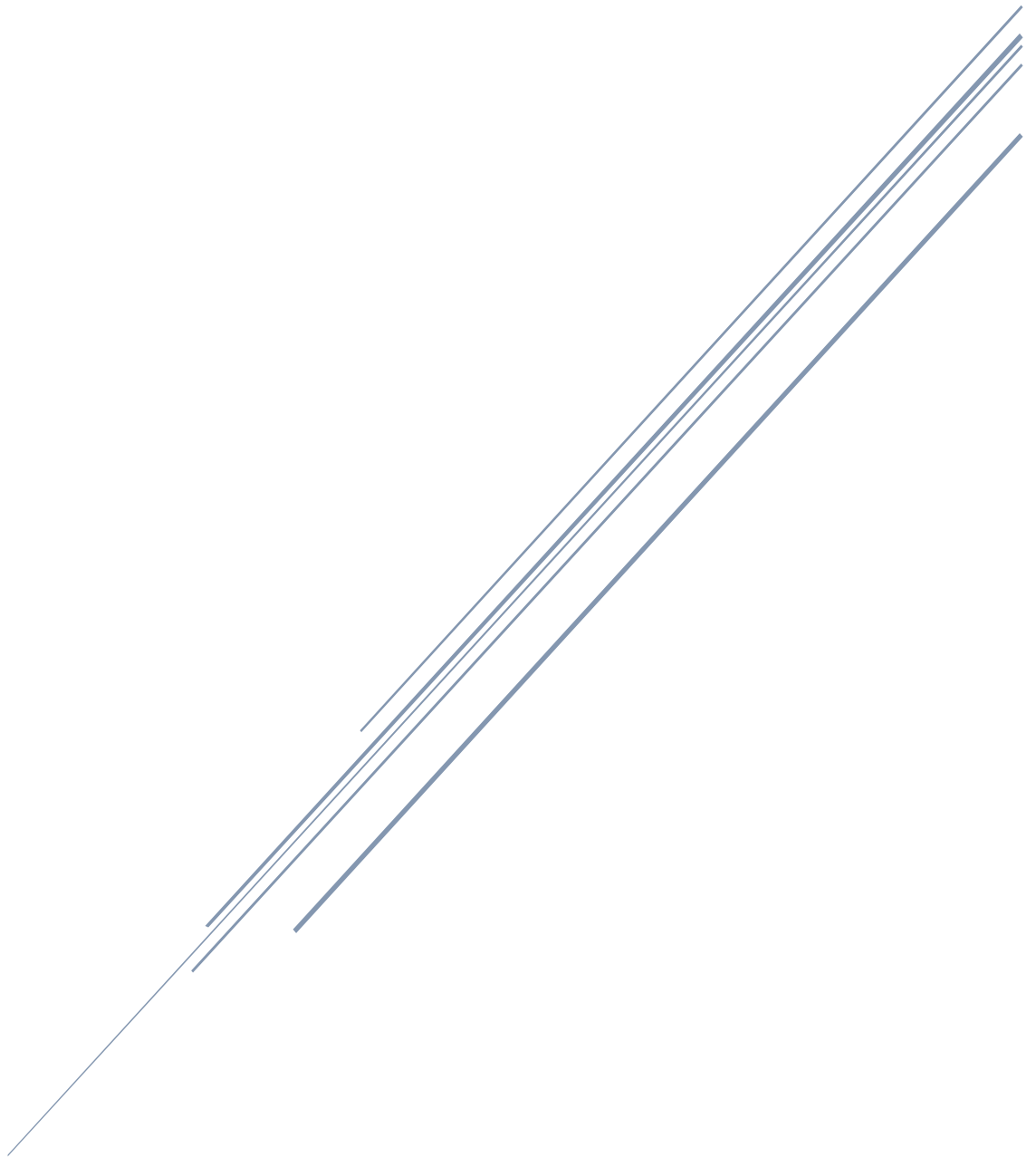


Agentic Financial Advisor Bot for Investment Strategies

Preliminary report



University of London
CM3070 Final Project

Abstract

The financial market presents a challenging environment for investors due to its volatility, high-dimensional data, and overwhelming information flow. This report introduces the design and development of an Agentic Financial Advisor Bot aimed at democratizing access to personalized, transparent investment strategies. The system integrates real-time financial data ingestion, lightweight forecasting via reservoir computing, and decision-making through deep reinforcement learning (FinRL). It further incorporates a natural language interface powered by Qwen3, enabling users to receive investment advice in conversational form. Built within the lightweight agentic framework Smolagents, the bot operates as an explainable, modular, and adaptable portfolio management assistant. Evaluation through backtesting and user feedback will assess the bot's effectiveness in improving investment outcomes and user trust. This project demonstrates how emerging AI technologies can be orchestrated to create accessible, intelligent financial tools for retail investors.

Contents

1.	Introduction.....	1
2.	Project Framework	2
3.	Motivation	3
4.	Literature Review.....	4
4.1.	Forecasting Approaches	4
4.2.	Deep Reinforcement Learning in Trading.....	5
4.2.1.	TD3 for Portfolio Optimization	5
4.2.2.	DDPG for Stock Trading	5
4.3.	Benchmarking and Data Environments.....	5
4.3.1.	FinRL-Meta: Modular Environments and Benchmarks	6
4.3.2.	Dynamic Datasets for Continual FinRL Training.....	6
4.4.	Multimodal Learning for Movement Prediction	7
4.5.	Comparative Analysis and Research Gaps.....	7
4.5.1.	Comparative Overview	8
4.5.2.	Research Gaps and Open Questions	8
4.6.	Conclusion	9
5.	Project Design.....	10
5.1.	Domain and Users	10
5.2.	Design Justification	10
5.3.	Project Structure	10
5.4.	Technologies and Methods	12
5.5.	Work Plan	12
5.6.	Evaluation Plan	14
5.7.	Contingency Plan	15
6.	Feature Prototype	16
6.1.	Forecasting with Reservoir Computing	16
6.2.	Reinforcement Learning with FinRL	18
6.3.	Natural Language Output with Smolagents + Qwen3.....	20
6.4.	Limitations and Planned Enhancements.....	21
7.	Conclusion	23
8.	References	24

1. Introduction

The financial market is a complex, dynamic environment characterized by high volatility, non-stationary signals, and a low signal-to-noise ratio. Accurately predicting asset prices or optimizing investment strategies remains a formidable challenge, attracting the attention of both researchers and practitioners. Traditional models like ARIMA and linear regressions often fall short in capturing nonlinear patterns and adapting to new market conditions.

This report outlines the design and development of an Agentic Financial Advisor Bot, built to support active portfolio management through a combination of financial data processing, predictive modelling, reinforcement learning and natural language interfaces. The system is designed to be adaptable, explainable, and lightweight, providing accessible investment recommendations with a clear rationale and transparency.

2. Project Framework

The project is based on the “Financial Advisor Bot” template. The main objective is to perform active portfolio management. To meet this goal, the system integrates:

- **Data Sources:** Live financial data from Yahoo Finance API, Bloomberg API, and CCXT.
- **Predictive Models:** Predictive analytics via reservoir computing.
- **Reinforcement Learning Environment:** FinRL for strategic decision-making.
- **Language Models:** Qwen3 for generating human-readable investment advice.
- **Agentic Framework:** Smolagents which is a minimalist lightweight Python framework for composing agentic systems using natural language to call functions.
- **Evaluation:** Backtesting for historical comparison and accuracy assessment.

This stack allows the system to perform daily predictions, plan for investment strategy, communicate them clearly, and evaluate performance.

3. Motivation

The financial decision-making landscape is characterized by several key user pain points:

- **Information Overload:** Users face an overwhelming volume of market data.
- **Expensive Advisory Services:** Traditional financial advisors are costly.
- **Limitations of Existing Tools:** Many tools are passive and lack personalization.

Users are seeking investment tools that are simple to use, offer clear transparency, and can adapt to changing market conditions. This project aims to address those needs with an intelligent, user-friendly agentic system.

4. Literature Review

In response, the recent surge in machine learning, particularly **deep learning** and **reinforcement learning (RL)**, has opened new avenues for financial forecasting and trading automation. These approaches bring the potential to model intricate temporal dependencies, adapt through online learning, and incorporate high-dimensional, multimodal data sources such as financial news, social media sentiment, and real-time prices.

This review synthesizes six papers that span three emerging fronts in financial AI:

1. **Time series forecasting** using lightweight neural architectures.
2. **Portfolio management and trading** using deep reinforcement learning.
3. **Data-centric and benchmark-driven platforms** for developing and evaluating financial AI agents.

Together, these works represent the convergence of model innovation, infrastructure development, and data integration—setting the stage for robust, adaptive, and scalable financial decision-making systems.

4.1. Forecasting Approaches

Traditional time series forecasting in finance has heavily relied on statistical models such as ARIMA, which assume linearity and stationarity—assumptions often violated in real-world financial data. As a result, researchers have explored machine learning-based alternatives to capture nonlinear dependencies and better handle the stochastic nature of asset prices.

A notable contribution in this space is the use of **Reservoir Computing (RC)**, specifically **Echo State Networks (ESNs)**, for forecasting crude oil prices. In *“Forecasting Crude Oil Prices Using Reservoir Computing”*, Kumar (2023) demonstrates that RC can outperform deep learning models like Long Short-Term Memory (LSTM) in both prediction accuracy and runtime. Unlike LSTM, which requires extensive training of recurrent connections, ESNs use a randomly initialized dynamic reservoir and train only the output weights—making them computationally efficient. Although the paper focuses on crude oil, its price dynamics share similarities with stock markets, making it a relevant example.

The paper evaluates both models using a dataset of daily crude oil prices from 2010 to 2023. The RC model achieves a remarkably low mean absolute percentage error (MAPE) of 1.57%, significantly better than the LSTM's 27.69%. Moreover, the ESN's training time is only 1.1 seconds, compared to LSTM's 423 seconds. These results underscore the potential of RC as a lightweight and effective solution for univariate financial time series forecasting.

While RC models offer speed and accuracy, they are primarily suited to single-asset prediction tasks and may lack the flexibility of more complex models when dealing with multivariate or multimodal data. Nonetheless, they provide a strong baseline for environments requiring fast, interpretable, and resource-efficient models.

4.2. Deep Reinforcement Learning in Trading

While forecasting models focus on predicting future prices or movements, **reinforcement learning (RL)** offers a more action-oriented framework—learning how to make decisions that maximize returns over time. In financial contexts, RL has been increasingly applied to **portfolio optimization**, **asset allocation**, and **automated trading**. Two prominent RL algorithms—**TD3** and **DDPG**—are explored in the reviewed literature, offering complementary insights into practical FinRL applications.

4.2.1. TD3 for Portfolio Optimization

In “*Deep Reinforcement Learning for Portfolio Selection*”, Jiang et al. propose an enhanced TD3 (Twin Delayed Deep Deterministic Policy Gradient) model that incorporates investor-specific constraints, including **risk aversion** and **transaction costs**. The agent is trained to optimise a utility function inspired by Markowitz’s mean-variance theory while interacting with a simulated market environment consisting of DJIA and S&P100 stocks.

Their empirical results demonstrate that the TD3 agent achieves **annual returns of up to 42%** and **Sharpe ratios exceeding 1.4**, outperforming both traditional methods (e.g., min-variance portfolios) and other DRL baselines (PPO, DDPG). The model is particularly well-suited to **high-dimensional portfolios**, where traditional models struggle due to noise and complexity.

4.2.2. DDPG for Stock Trading

In a complementary study, Liu et al. present “*Practical Deep Reinforcement Learning Approach for Stock Trading*”, which employs DDPG (Deep Deterministic Policy Gradient) to train an agent that learns **online** as it trades. The environment includes 30 DJIA stocks and spans a dataset from 2009 to 2018. The agent’s objective is to maximize cumulative portfolio value while navigating market dynamics.

The DDPG-based agent significantly outperforms both the DJIA index and a min-variance benchmark, achieving an **annualized return of 25.87%** and a **Sharpe ratio of 1.79**. The key advantage of DDPG lies in its ability to operate in **continuous action spaces**, allowing fine-grained control over position sizes in each asset.

Together, these studies illustrate the growing maturity of deep RL in financial applications. While DDPG is effective for single-agent trading scenarios, TD3 excels in constrained, risk-aware portfolio management. Both models highlight the importance of **reward design**, **training stability**, and **realistic environment simulation** in the successful deployment of RL agents.

4.3. Benchmarking and Data Environments

While algorithmic advances in financial reinforcement learning (FinRL) are promising, they are often evaluated on non-standard datasets or with custom environments, making

comparability and reproducibility difficult. Two recent contributions address this gap by providing **benchmarking platforms** and **dynamic data pipelines** that enable realistic and standardized evaluation of FinRL models.

4.3.1. FinRL-Meta: Modular Environments and Benchmarks

In “*FinRL-Meta: Market Environments and Benchmarks for Data-Driven Financial Reinforcement Learning*”, Liu et al. introduce a comprehensive **RLOps platform** that automates the end-to-end process of building and evaluating FinRL agents. FinRL-Meta supports a wide range of financial tasks—stock trading, portfolio management, multi-agent liquidation—implemented as **OpenAI Gym-style environments**.

FinRL-Meta leverages the **DataOps paradigm** to automate data ingestion, cleaning, feature engineering, and simulation setup. It supports data from diverse markets (stocks, ETFs, crypto, forex) and integrates **technical, fundamental, sentiment, and alternative** features (e.g., ESG, patents, social media).

Importantly, the platform reproduces benchmarks for common DRL algorithms such as **DDPG, PPO, and A2C**, providing reproducible baselines for academic and industrial research. Ensemble strategies further enhance performance, with ensemble-PPO+A2C+DDPG achieving a **Sharpe ratio of 1.53** and outperforming single agents and traditional indices.

4.3.2. Dynamic Datasets for Continual FinRL Training

Extending FinRL-Meta, “*Dynamic Datasets and Market Environments for Financial Reinforcement Learning*” addresses the **static data limitation** in RL training. Static datasets can lead to overfitting and poor generalization. This work proposes a **rolling-window training-testing-trading pipeline** that allows agents to continuously learn and adapt as new data arrives.

The framework supports **real-time data ingestion** from 30+ financial sources and is compatible with major RL libraries (Stable-Baselines3, RLlib, ElegantRL). It also includes **minute-level paper trading** through integration with brokers like Alpaca.

The dynamic pipeline enhances realism, reduces survivorship bias, and supports **curriculum learning**—gradually increasing task complexity to improve model generalization. The use of ensemble agents in this framework consistently outperforms single-policy models and traditional baselines.

Together, these two works signal a shift from isolated model development toward **system-level thinking** in FinRL: robust pipelines, standardized benchmarks, and real-world-ready infrastructure. They are essential for bridging the simulation-to-reality gap in financial AI research.

4.4. Multimodal Learning for Movement Prediction

In financial markets, asset movements are often influenced by **unstructured external signals**—such as breaking news, investor sentiment, and public discussions on social media. While price-based models capture historical trends, they often miss real-time reactions to external events. To address this, researchers have turned to **multimodal learning**, integrating textual and numerical data for improved predictive power.

A pioneering effort in this domain is “*Stock Movement Prediction from Tweets and Historical Prices*” by Xu and Cohen, where the authors introduce **StockNet**, a deep **variational generative model** that fuses **tweets and historical price data** to predict binary stock movements (up/down).

Unlike traditional discriminative approaches, StockNet models the **latent market dynamics** using **recurrent, continuous latent variables** and trains them using **variational inference**. The model includes three major components:

1. A **Market Information Encoder** that processes tweet embeddings and price changes,
2. A **Variational Movement Decoder** that captures stochastic dependencies,
3. An **Attentive Temporal Auxiliary module** that leverages prior movement predictions to improve temporal coherence.

The model is trained on a dataset covering 88 U.S. stocks across two years (2014–2016), combining Twitter messages and Yahoo Finance price data. It achieves **58.2% accuracy**, outperforming all baselines—including ARIMA, random forests, and state-of-the-art NLP models like HAN. Notably, the generative model also produces a higher **Matthews Correlation Coefficient (MCC)**, indicating stronger prediction reliability in imbalanced datasets.

StockNet's performance demonstrates the **value of combining social sentiment with technical indicators**. It also shows that deep generative models—traditionally used in language modelling—can effectively capture the uncertainty and chaos inherent in financial markets.

However, the model focuses solely on **binary movement prediction**, without addressing portfolio-level decision-making or capital allocation. Nonetheless, it lays important groundwork for future FinRL agents that incorporate **multimodal market intelligence** into their policy learning.

4.5. Comparative Analysis and Research Gaps

The six reviewed papers cover a broad and complementary range of financial AI research—from forecasting and trading strategies to infrastructure and data integration. Collectively, they illustrate how modern techniques in **deep learning, reinforcement learning**, and

generative modelling are transforming the way we approach financial prediction and decision-making.

4.5.1. Comparative Overview

Category	Paper(s)	Key Strengths	Limitations
Forecasting	Reservoir Computing [Kumar 2023]	High accuracy, low latency, simple implementation	Single-asset only; no adaptive learning
Trading Agents	DDPG [Liu et al. 2018], TD3 [Jiang et al. 2024]	Optimised capital allocation, risk sensitivity, outperform traditional methods	Require careful tuning; sensitive to data realism
Infrastructure	FinRL-Meta [Liu et al. 2022], Dynamic Datasets [Liu et al. 2023]	Reproducibility, modularity, live data support, benchmark environments	Require computational infrastructure; steep learning curve
Multimodal Modelling	StockNet [Xu and Cohen 2018]	Incorporates sentiment & price, generative model with temporal structure	Binary-only prediction, not integrated with RL

These approaches vary in their **scope** (single asset vs. portfolio), **output** (prediction vs. policy), and **data modalities** (prices, sentiment, fundamentals). However, all point to a need for **more robust, adaptive, and explainable AI systems** in finance.

4.5.2. Research Gaps and Open Questions

Based on the synthesis, several research gaps emerge:

- **Hybrid Modelling:** Most models focus either on prediction (StockNet, RC) or decision-making (TD3, DDPG). Bridging the two—e.g., by integrating StockNet-like sentiment encoders into DRL agents—remains largely unexplored.
- **Explainability:** Despite high performance, these models lack interpretability. In finance, black-box systems face resistance due to regulatory and fiduciary concerns. Future work should incorporate explainable RL and feature attribution.
- **Multi-Agent Systems:** Financial markets are inherently multi-agent. Yet, most reviewed works train single-agent models. Competitive and cooperative multi-agent RL could better simulate market dynamics.

- **Cross-Modal Reasoning:** StockNet shows the potential of integrating tweets and prices, but further integration with **news**, **macroeconomic indicators**, and **alternative data** (e.g., ESG scores, patents) could enhance robustness.
- **Real-Time Deployment and RLOps:** While Papers 4 and 5 address infrastructure, few models are deployed in real-time or evaluated in live markets. Bridging the gap between research prototypes and production systems remains a major challenge.

These research gaps provide promising directions for advancing financial AI systems that are not only accurate, but also robust, transparent, and ready for real-world deployment.

4.6. Conclusion

This literature review has explored six influential contributions at the intersection of deep learning, reinforcement learning, and financial prediction. The reviewed works highlight significant progress along three major fronts:

1. **Forecasting innovations**, exemplified by Reservoir Computing, offer efficient alternatives to deep models like LSTM for fast and accurate asset prediction.
2. **Reinforcement learning applications**, such as TD3 and DDPG, have demonstrated the capability of AI agents to make risk-sensitive portfolio decisions and outperform traditional benchmarks.
3. **Data-centric and infrastructure-focused advancements**, as seen in FinRL-Meta and dynamic datasets, are pushing the field toward reproducibility, standardization, and real-time deployment.

Additionally, the integration of **multimodal data sources**, as pioneered by StockNet, illustrates the growing importance of combining sentiment and numerical indicators to better understand market behaviour.

Despite these advancements, challenges remain—particularly in model explainability, online adaptation, and the development of hybrid systems that can both predict and act. Future research should focus on bridging these gaps through cross-disciplinary approaches, combining robust machine learning techniques with domain expertise in finance.

As the field matures, we move closer to building **transparent, adaptive, and trustworthy AI systems** capable of navigating the complexity and volatility of modern financial markets.

5. Project Design

The Agentic Financial Advisor Bot is an AI-powered tool designed to provide intelligent, explainable, and adaptive financial advice for investors. It combines real-time market data processing, predictive modelling, reinforcement learning, and natural language generation to deliver clear and actionable investment strategies. The system enables users to interact with the bot conversationally, receiving customized investment guidance while maintaining transparency and control.

5.1. Domain and Users

Domain: Financial technology and personal investment advisory.

Users:

- Retail investors seeking low-cost, accessible investment advice.
- Beginner investors overwhelmed by data and lacking financial expertise.
- Users looking for a transparent alternative to black-box robo-advisors.

The project addresses the intersection of personal finance and artificial intelligence, providing an intelligent interface for individuals (e.g., “A 29-year-old retail investor using Robinhood”) to manage their investments actively without incurring the high costs of traditional advisors.

5.2. Design Justification

The project is designed to address three main user challenges:

- **Information Overload:** Users are bombarded with excessive and often conflicting financial information.
- **Advisor Costs:** Traditional financial advisors remain unaffordable for many retail investors.
- **Lack of Adaptability in Tools:** Most existing tools are passive and lack explainability.

The design choices—real-time data integration, predictive modelling, reinforcement learning for strategy selection, and natural language generation—are directly aligned with these user needs. The agentic approach ensures explainability and adaptability, while leveraging existing APIs and LLMs enhances accessibility.

5.3. Project Structure

The system is designed as a modular, agentic pipeline consisting of six core components. Each module handles a specific function in the forecasting, decision-making, and communication process:

1. Perception:

- Retrieves and preprocesses live or historical financial data using APIs such as Yahoo Finance. This includes normalization, resampling, and feature engineering to prepare the input for downstream models.

2. Reasoning:

- Applies predictive models—such as Reservoir Computing (RC)—to forecast short-term price movements or trends based on recent market behaviour.

3. Action Selection:

- Uses reinforcement learning, implemented via the FinRL framework, to determine optimal portfolio allocations based on predicted market conditions and observed features.

4. Natural Language Reporting:

- Converts technical outputs into user-facing, conversational explanations using the Qwen3 language model. This module provides transparency and improves accessibility for non-expert users.

5. Self-Evaluation:

- Logs predictions and actions for each episode and evaluates performance using backtesting tools such as pyfolio. This helps refine policy model.

6. Agentic Framework:

- Smolagents is used to coordinate the components via natural language-based function calling. It enables flexible orchestration of forecasting, strategy, and reporting modules in response to user queries.

Control Flow and Interaction Design

The bot operates in an agentic fashion:

- The **user interacts with the LLM interface** (Qwen3), asking questions or requesting investment recommendations in natural language.
- **Smolagents parses the user intent** and determines whether additional tools are needed (e.g., fetching market data, generating forecasts, calling the RL policy).
- The required modules are then triggered in sequence. For example, a user asking “What should I do with AAPL today?” results in:
 - pulling recent AAPL data (Perception),
 - generating a prediction (Reasoning),
 - running the RL policy (Action),

- and returning a human-readable recommendation (Natural Language Reporting).

This design supports **modularity**, **interpretability**, and **flexibility**, enabling future upgrades (e.g., swapping RC with a Transformer model, or Qwen3 with other LLM model like ChatGPT) without altering the core architecture.

5.4. Technologies and Methods

- **Reservoir Computing:** Time-series prediction model with low computational cost.
- **FinRL:** Reinforcement learning framework for financial trading with built in Reinforcement Learning and backtesting feature.
- **Qwen3:** A family of open-source large language models, with the lightweight variant used in this project.
- **Smolagents:** Lightweight agentic system framework.
- **Yahoo Finance, Bloomberg, CCXT APIs:** Live financial data.

5.5. Work Plan

To accelerate early development, simpler models such as A2C for reinforcement learning and standard reservoir computing for forecasting are used in the initial prototype. However, these are not intended for the final version. The system is designed to allow modular upgrades; future iterations will incorporate more advanced components such as DDPG or PPO for trading and Next-Generation Reservoir Computing (NG-RC) for forecasting [Gauthier et al., 2021]. As part of the reinforcement learning upgrade, the final model may incorporate ideas from recent work on stigmergic multi-agent learning, particularly mechanisms for indirect coordination and federated training. This is inspired by [Xu et al., 2020], and would enable more robust portfolio strategies under dynamic market conditions.

The following Gantt Chart outlines the planned phases of the project. The primary goal is to develop a working pipeline by Week 12. Testing of each individual component is scheduled for Week 11. The system will use Smolagents to coordinate RC forecasts, FinRL-generated strategies, and Qwen3 explanations into a cohesive user interaction loop.

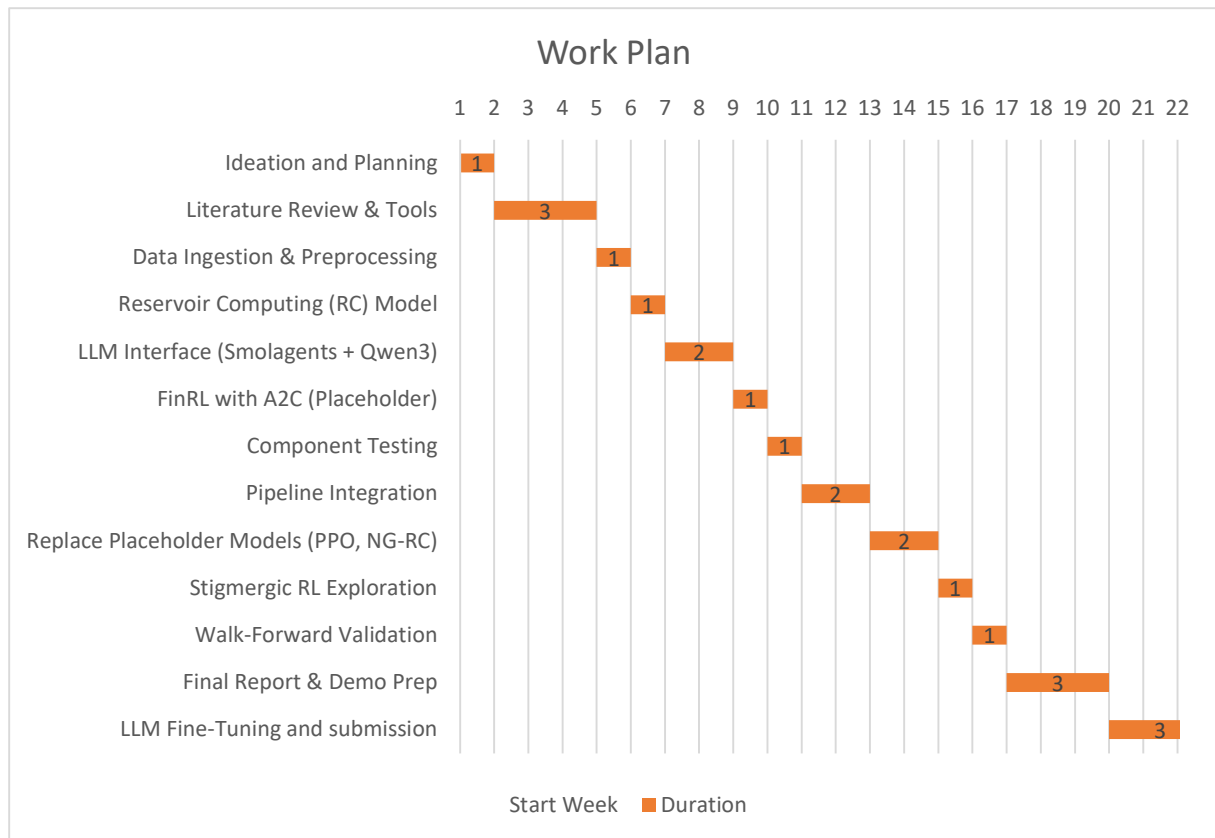


Figure 1. Work Plan Gantt Chart

Week	Task
1	Ideation and Planning
2-4	Literature review, finalize tools and technologies
5	Prototype data ingestion and preprocessing module
6	Develop and Test RC Models (based on Kumar, 2023)
7-8	Connect LLM (Qwen3) with Agentic Framework (Smolagents) for NL Output
9	Test FinRL for Action Strategy Generation and Backtesting
10	Internal Testing of Individual Components and preliminary report submission
11-12	Integrate Components (RC + FinRL + LLM) into a Unified Pipeline and explore possible user-friendly web interface
13	Replace Placeholder Models (e.g., A2C → PPO, RC → NG-RC)
14-15	Explore Stigmergic RL Architecture (based on Xu et al., 2020)

16	Evaluate System Performance (Quantitative & Qualitative Metrics), Stress Testing with Market Volatility Scenarios
17-19	Prepare Final Report, Slides, and Video Presentation
20-22	LLM fine-tuning, polishing and final submission

5.6. Evaluation Plan

The evaluation of the prototype system focuses on three core modules: forecasting, reinforcement learning-based decision-making, and natural language interaction. Each is tested independently due to the modular design, with metrics tailored to the functionality of each component.

Forecasting Module (Reservoir Computing)

The RC model is evaluated on its ability to predict the next-day closing price of AAPL using historical data. The evaluation is based on a 100-step forecast horizon, using the following metrics:

- **Mean Absolute Error (MAE)**
- **Root Mean Squared Error (RMSE)**
- **Mean Absolute Percentage Error (MAPE)**
- **R-squared (R^2)**
- **Normalized RMSE (based on mean and range)**

A baseline performance is compared against a naive predictor (e.g., yesterday's price = today's prediction). The model is considered effective if it significantly outperforms the baseline in stable market conditions and degrades predictably in volatile regimes.

Reinforcement Learning Module (FinRL + A2C)

The A2C agent is evaluated using **backtesting** on unseen data (July 2020 to October 2021) in a simulated trading environment. The following financial and performance metrics may be used:

- **Cumulative return**
- **Sharpe ratio**
- **Maximum drawdown**
- **Volatility of returns**
- **Portfolio allocation stability**

Comparisons are made against a passive buy-and-hold strategy of the DOW30 index. Due to A2C's placeholder status, the focus is on functional correctness and responsiveness to market signals rather than peak performance.

Natural Language Module (Smolagents + Qwen3)

This component is evaluated qualitatively by:

- Demonstrating task routing accuracy via Smolagents
- Verifying that queries such as *"What should I do with AAPL today?"* return coherent and relevant responses
- Checking consistency and tone of Qwen3 output under various inputs

In later stages, user studies or feedback from simulated queries could be used to assess response helpfulness and trustworthiness.

Evaluation Limitations

- Integration between modules is not yet complete; evaluations are conducted in isolation.
- Backtesting does not currently simulate transaction costs, slippage, or market impact.
- No walk-forward validation or cross-validation has been applied yet.

Future Improvements

The evaluation plan will be extended to include:

- Rolling retraining for forecasting models
- Real-time testing on live or delayed market data
- Task success scoring for the LLM based on user satisfaction
- Comparative testing across multiple RL models (e.g., PPO, TD3) under identical simulated conditions

5.7. Contingency Plan

- If LLM integration fails, fallback to templated explanations based on model output.
- If RL model training underperforms, fall back to a rule-based or pre-trained policy using historical averages or simple momentum indicators.
- Performing backtesting in real time is difficult and require large time window. We would therefore use historic datasets for testing and demonstrations.

6. Feature Prototype

This section presents a modular demonstration of the key components developed for the Agentic Financial Advisor Bot. While full system integration is still underway, each module has been implemented and tested individually to validate its functionality and design.

The prototype currently consists of three core components:

1. **Forecasting Module:** A Reservoir Computing (RC) model trained on historical stock data to predict next-day prices. Its simplicity and speed make it well-suited for frequent and rapid retraining in volatile markets.
2. **Strategy Module:** A reinforcement learning agent implemented using the FinRL framework. Currently, a lightweight **Advantage Actor-Critic (A2C)** algorithm is used as a placeholder to test environment setup and action selection. **It is not intended for the final submission**, and will be replaced with a more advanced model in later iterations.
3. **Language Interface Module:** A natural language interface built using Smolagents and Qwen3, enabling users to interact with the system through conversational queries and receive readable, explainable advice.

Each component is designed to be lightweight, interpretable, and modular—allowing future upgrades and replacements without disrupting the entire pipeline. This section walks through the current implementation of each part, along with selected outputs, design motivations, and key limitations.

6.1. Forecasting with Reservoir Computing

We begin with a lightweight time-series forecasting module using **Echo State Networks (ESNs)**, a form of Reservoir Computing (RC). This approach was chosen for its:

- **Fast training speed:** only the output weights are trained via ridge regression,
- **Good performance on univariate financial data**, and
- **Simplicity in setup**, making it an effective baseline for real-time applications.

In this prototype, we focus on forecasting the next-day **closing price of AAPL (Apple Inc.)**, using historical price data spanning **from January 2, 2008 to June 12, 2025**. The model is trained on the **first 1000 normalised closing price values**, using a **window of 10 previous time steps as input**.

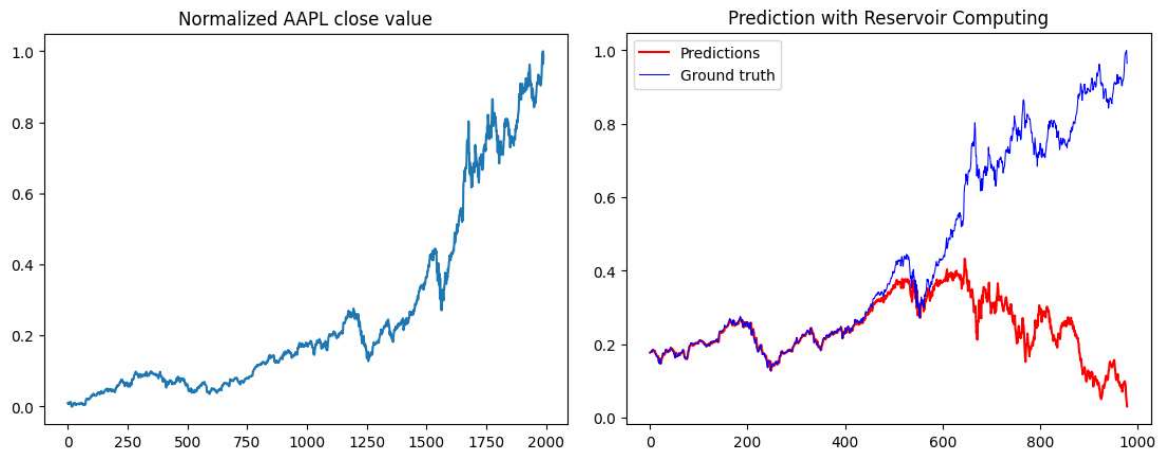


Figure 2.
Left: Normalised AAPL close value used to train and test RC model
Right: Actual vs. predicted AAPL prices using RC model.

The dataset was normalized to [0, 1] and partitioned into:

- **Training set:** 1000 time steps (2008–2011),
- **Evaluation:** following 900+ steps, extending into periods of significant volatility.

As shown in **Figure 2**, the RC model tracks the actual closing prices well for the first 400 prediction steps. However, its accuracy deteriorates significantly when AAPL enters a phase of rapid and uncharacteristic growth. This behaviour reflects a common limitation of static forecasting models—they struggle to generalize when the underlying data distribution shifts significantly, such as during **earnings surprises**, **macroeconomic shocks**, or **black swan events** like the COVID-19 pandemic.

To quantify performance, we evaluate the model using several standard error metrics:

- **Mean Absolute Error (MAE):** 0.2056
- **Root Mean Squared Error (RMSE):** 0.3467
- **Mean Absolute Percentage Error (MAPE):** 27.33%
- **R-squared (R^2):** -0.7052
- **Mean Squared Error (MSE):** 0.1202
- **Normalized RMSE (based on mean):** 0.7776
- **Normalized RMSE (based on max - min):** 0.3975

The **negative R^2 value** indicates that the model performs worse than a simple mean predictor in this evaluation window, especially during non-stationary periods. This further supports the need for **frequent retraining** or transitioning to **Next-Generation Reservoir Computing (NG-RC)** for improved adaptability and generalization under volatile market conditions.

6.2. Reinforcement Learning with FinRL

To automate portfolio allocation, we model the stock trading process as a **Markov Decision Process (MDP)** and optimise it using **Deep Reinforcement Learning (DRL)**. In this framework, the agent interacts with a simulated financial environment and learns to allocate capital across a portfolio to maximize cumulative returns over time. This model is based on the FinRL framework described in [Liu et al. 2022].

MDP Formulation

- **Action Space** ($a \in (-1, 1)$): The agent outputs a portfolio weight vector $[a_1, a_2, \dots, a_n]$, where a_i represents the proportion of capital allocated to the i^{th} asset. In this prototype, weights are constrained to $a_i \in [0, 1]$ (i.e., long-only), and $\sum a_i = 1$.
- **State Space**: The agent observes a set of financial indicators per asset, including Open, High, Low, Close prices, volume, and a suite of technical indicators (e.g., MACD, RSI, ADX). This mirrors the kind of market data a human investor would consider before making a trade.
- **Reward Function**: The reward is defined as the change in total portfolio value:
$$r(s, a, s') = v' - v,$$
where v and v' represent the portfolio values at states s and s' , respectively. This directly incentivizes the agent to increase portfolio value through its actions.
- **Environment**: The environment includes the **30 constituent stocks of the Dow Jones Industrial Average (DOW30)**. Historical stock data is sourced from the Yahoo Finance API.

Dataset and Preprocessing

- **Training period**: January 1, 2009 – July 1, 2020
- **Trading period (backtesting)**: July 1, 2020 – October 31, 2021
- **Assets**: All 30 stocks in the DOW30 index
- **Data source**: Yahoo Finance (via FinRL data pipeline)
- **Preprocessing**: Performed using FinRL's default pipeline, including:
 - Technical indicators: MACD, RSI, CCI, and ADX
 - Feature scaling and missing value handling

Preprocessing is handled entirely through FinRL's built-in pipeline to ensure reproducibility and consistency. No manual tuning was performed due to limited domain expertise in financial feature selection.

Model Implementation

- **Framework:** FinRL
- **Algorithm:** Advantage Actor Critic (A2C)
- **Episodes:** 50
- **Training Frequency:** Daily
- **Action granularity:** Continuous portfolio weights across 30 assets

The A2C algorithm is used as a **placeholder model** to validate the RL pipeline. It is a policy gradient method that separately learns a value function (critic) and an action policy (actor). Although computationally efficient, A2C may underperform on complex allocation tasks without further tuning.

Portfolio Allocation (Action Output)

During each time step, the agent outputs a **vector of portfolio weights** representing how much capital to allocate to each of the assets in DOW30. These weights sum to 1 and are constrained to be non-negative.

A sample allocation generated by the A2C agent is shown below:

	AAPL	AMGN	AXP	BA	CAT	CRM	CSCO	CVX	DIS	GS	...	MMM
date												
2020-07-01	0.035714	0.035714	0.035714	0.035714	0.035714	0.035714	0.035714	0.035714	0.035714	0.035714	...	0.035714
2020-07-02	0.039087	0.021217	0.036143	0.021217	0.041956	0.044596	0.021217	0.056761	0.021217	0.043837	...	0.029670
2020-07-06	0.039087	0.021217	0.036143	0.021217	0.041956	0.044596	0.021217	0.056761	0.021217	0.043837	...	0.029670
2020-07-07	0.039093	0.021209	0.036193	0.021209	0.041960	0.044606	0.021209	0.056802	0.021209	0.043838	...	0.029670
2020-07-08	0.039087	0.021217	0.036143	0.021217	0.041956	0.044596	0.021217	0.056761	0.021217	0.043837	...	0.029670

Figure 3. Sample portfolio allocation by A2C agent.

This vector is the **core action** taken by the agent, and it is used to simulate market interaction in the environment. At the next time step, the agent observes the market response (e.g., price changes) and receives a reward based on the updated portfolio value.

Backtesting

Backtesting plays a critical role in evaluating the performance and robustness of an automated trading strategy. It allows us to assess how well the agent would have performed in historical market conditions—before risking real capital.

To reduce human error and ensure consistency, we use an **automated backtesting workflow**, powered by the **Quantopian pyfolio package**. pyfolio provides a rich set of diagnostic tools and visualizations, including:

- Cumulative returns
- Daily returns
- Rolling Sharpe ratio
- Drawdown periods
- Exposure analysis
- Tear sheets for overall performance evaluation

The graph below is a comparison between our model and baseline model from pyfolio

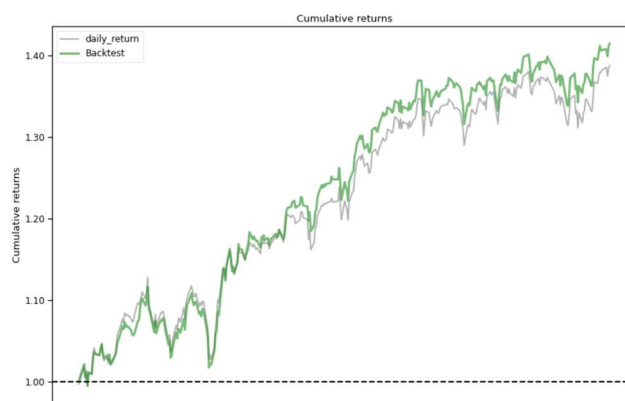


Figure 4. Backtesting performance compared to baseline from pyfolio.

6.3. Natural Language Output with Smolagents + Qwen3

To complete the feedback loop between the system and the user, we integrate a natural language interface powered by **Smolagents** and **Qwen3**. This allows users to interact with the system using natural language queries and receive clear, human-readable explanations of forecasts, strategy decisions, and portfolio recommendations.

Architecture and Design

The language module consists of two main parts:

- **Smolagents Framework:** Handles function routing. It parses the user’s natural language query, identifies the target function (e.g., “get latest data”, “make prediction”, “tell me what to invest in”), and routes it to the appropriate functions.

- **Qwen3 LLM:** Converts structured outputs (e.g., actions, recommendations) into fluent natural language responses. It provides a conversational layer between the agent's logic and the user.

This setup allows the model to behave like a virtual financial assistant, generating explanations based on model output and user queries, making the interaction less intimidating.

A simple example of the system's input-output flow is shown below:

Assistant: </think>

Hey there! I'm Qwen, your friendly and witty stock investing buddy. I can help you with DOW30 insights, but I need to know which stock you're interested in to give you the best advice. Let's keep things light and fun-maybe I can even throw in a joke about market trends! 🤖💡

Enter a message (type 'exit' to quit): could you give me the latest data for Apple and American Express

🔍 Agent is handling your request...

New run

could you give me the latest data for Apple and American Express

TransformersModel - Qwen/Qwen3-1.7B

Step 1

— Executing parsed code: —

```
apple_data = fetch_latest_data("AAPL")
american_express_data = fetch_latest_data("WBA")
final_answer(f"Apple: {apple_data}, American Express: {american_express_data}")
```

Out - Final answer: Apple: On 2021-11-30, AAPL opened at \$157.03, reached a high of \$162.46, a low of \$156.96, and closed at \$162.24 with a volume of 174,048,100.would you like to have next day close prediction or trading strategies?, American Express: On 2021-11-30, WBA opened at \$37.96, reached a high of \$38.33, a low of \$36.60, and closed at \$36.93 with a volume of 13,238,100.would you like to have next day close prediction or trading strategies?

[Step 1: Duration 203.00 seconds | Input tokens: 2,254 | Output tokens: 76]

Assistant: Apple: On 2021-11-30, AAPL opened at \$157.03, reached a high of \$162.46, a low of \$156.96, and closed at \$162.24 with a volume of 174,048,100 would you like to have next day close prediction or trading strategies?, American Express: On 2021-11-30, WBA opened at \$37.96, reached a high of \$38.33 a low of \$36.60, and closed at \$36.93 with a volume of 13,238,100.would you like to have next day close prediction or trading strategies?

Figure 5. Sample natural language interaction with the system.

This interaction demonstrates how the system converts technical outputs into accessible, user-friendly advice. However, this is far from perfect as the model might hallucinate or confuse user's request. In this example, the model confuses WBA (Walgreens Boots Alliance) and AXP (American Express).

6.4. Limitations and Planned Enhancements

While the prototype demonstrates the individual components of the Agentic Financial Advisor Bot, several limitations remain that will be addressed in future development stages:

Model Limitations

- The **Reservoir Computing (RC)** model, while lightweight and fast, degrades rapidly under distribution shifts. It lacks multivariate inputs and cannot adapt to regime changes without frequent retraining.
- The **A2C reinforcement learning agent** serves as a placeholder and lacks robustness for complex allocation tasks. It currently does not incorporate transaction costs, risk constraints, or exposure limits.

- The **Qwen3 language model** has not been fine-tuned on financial dialogue, and its outputs may be generic or imprecise in some contexts.

Evaluation and Testing Gaps

- No **walk-forward validation** or rolling-window backtesting has been implemented.
- Performance metrics like Sharpe ratio and drawdown are not yet integrated into the reward function.
- The system has only been tested on historical data, with no real-time inference or user trials.

Integration Status

- Components are currently tested in isolation. The end-to-end pipeline is still under development.
- The natural language interface cannot yet trigger real-time data queries or model calls autonomously.

Planned Enhancements

To address the limitations above, the following improvements are planned:

- **Upgrade Strategy Module:** Replace A2C with more stable algorithms such as **TD3** or **PPO** for better control in continuous action spaces.
- **Stigmergic RL Exploration:** Incorporate **stigmergic independent reinforcement learning** for multi-agent portfolio collaboration and distributed decision-making.
- **Improve Forecasting:** Implement **Next-Generation Reservoir Computing (NG-RC)** for more interpretable and adaptive forecasting.
- **Risk-Aware Reward Functions:** Redesign the reward to include **Sharpe ratio**, **drawdown penalties**, and **transaction cost models**.
- **LLM Fine-Tuning:** Fine-tune Qwen3 on a dataset of financial queries and reports for more relevant and trustworthy responses.
- **Pipeline Integration:** Use Smolagents to orchestrate RC, FinRL, and Qwen3 in a single callable chain.
- **Deployment & UI:** Create a web-based interface or CLI assistant for user interaction and daily portfolio suggestions.

7. Conclusion

This report has presented the modular design, technical framework, and prototype implementation of an Agentic Financial Advisor Bot. By combining reservoir computing for forecasting, reinforcement learning for strategy generation, and large language models for explainability, the system provides a lightweight and transparent decision-making pipeline for personal investment. Future work will include improving robustness under market volatility, fine-tuning the LLM interface, and integrating stigmergic multi-agent methods for enhanced adaptability.

8. References

1. Kumar, K. (2023). *Forecasting crude oil prices using reservoir computing models*. arXiv preprint arXiv:2306.03052. <https://arxiv.org/abs/2306.03052>
2. Jiang, Y., Olmo, J., & Atwi, M. (2024). *Deep reinforcement learning for portfolio selection*. Journal of Econometrics. <https://doi.org/10.1016/j.jeconom.2024.105641>
3. Liu, B., Wang, Y., Shao, Y., Zhu, Y., Fu, J., Chen, Y., & Li, X. (2018). *Practical deep reinforcement learning approach for stock trading*. arXiv preprint arXiv:1811.07522. <https://arxiv.org/abs/1811.07522>
4. Liu, B., Zhang, J., Chen, Y., Wang, Y., & Li, X. (2022). *FinRL-Meta: Market environments and benchmarks for data-driven financial reinforcement learning*. arXiv preprint arXiv:2211.03107. <https://arxiv.org/abs/2211.03107>
5. Liu, B., Zhang, J., Xie, Y., Chen, Y., Wang, Y., & Li, X. (2023). *Dynamic datasets and market environments for financial reinforcement learning*. arXiv preprint arXiv:2304.13174. <https://arxiv.org/abs/2304.13174>
6. Xu, Y., & Cohen, S. B. (2018). *Stock movement prediction from tweets and historical prices*. Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL 2018). <https://github.com/yumoxu/stocknet-dataset>
7. Gauthier, D.J., Bollt, E., Griffith, A. *et al.* Next generation reservoir computing. *Nat Commun* **12**, 5564 (2021). <https://doi.org/10.1038/s41467-021-25801-2>
8. Xu, X., Li, R., Zhao, Z., & Zhang, H. (2021). *Stigmergic independent reinforcement learning for multi-agent collaboration*. arXiv preprint arXiv:1911.12504. <https://arxiv.org/abs/1911.12504>
9. Liu, X.-Y., Yang, H., Chen, Q., Zhang, R., Yang, L., Xiao, B., & Wang, C. D. (2022). *FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance*. arXiv preprint arXiv:2011.09607. <https://arxiv.org/abs/2011.09607>